

Navigating The Future of Online Shopping

Ecommerce Analysis Using Python and SQL

-Shreyash Kale



About Company

This dataset captures transactional data from a multi-category e-commerce platform that connects customers and sellers across diverse regions. The platform supports a broad product catalog and flexible payment options, including installment plans.

As competition in the e-commerce landscape intensifies, analyzing customer behavior, sales performance, and seller contributions becomes essential for driving sustainable business growth.

Tools And Technologies

- SQL (MySQL) – Data Extraction, Querying
- Python (Pandas, Matplotlib) – Data Analysis, Visualization
- VsCode – Implementation
- Workflow – Data Cleaning > Analysis > Visualization > Insights

Dataset Overview

File	Description	Rows
customers.csv	Customer demographic details	99,441
sellers.csv	Seller information	3,095
orders.csv	Order history and details	99,441
order_items.csv	Order items details	112,650
payments.csv	Payment details	103,886
products.csv	Product details	32,951
geolocation.csv	Geolocation details	1,000,163

Insights - Basics

Objective: Extract fundamental insights from the dataset.

- Unique cities where customers are located – 4119 cities
- Number of orders placed in 2017 – 45,101 orders
- Total sales per category – health & Beauty Rs 1.25m
- Percentage of orders paid in installments – 49.42%
- Number of customers from each state – SP leads with 41,746

UNIQUE CITIES WHERE CUSTOMERS ARE LOCATED

python

```
customers['customer_city'].unique()
```

```
array(['franca', 'sao bernardo do campo', 'sao paulo', ...,  
      'monte bonito', 'sao rafael', 'eugenio de castro'], dtype=object)
```

Sql

```
-- Q1: List all unique cities where customers are located  
SELECT DISTINCT customer_city  
FROM customers  
ORDER BY customer_city;
```

```
=====
```

```
Q1: List all unique cities where customers are located
```

```
=====
```

```
customer_city  
franca  
sao bernardo do campo  
sao paulo  
mogi das cruzeiras  
campinas  
jaraguá do sul
```

COUNT OF NO. OF ORDERS PLACED IN 2017

python

```
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])  
orders_2017 = orders[orders['order_purchase_timestamp'].dt.year == 2017]['order_id'].nunique()  
orders_2017
```

45101

Sql

```
-- Q2: Count the number of orders placed in 2017  
✓ SELECT COUNT(*) AS total_orders_2017  
FROM orders  
WHERE year(order_purchase_timestamp) = '2017';
```

```
=====
```

Q2: Count the number of orders placed in 2017	
=====	
total_orders_2017	
45101	

TOTAL SALES PER CATEGORY

python

```
temp = order_items.merge(
    products[['product_id', 'product_category']],
    on='product_id',
    how='inner'
)
temp['product_category'] = temp['product_category'].fillna('Unknown')
category_sales = temp.groupby('product_category')['price'].sum().reset_index()
category_sales.rename(columns=[
    'product_category': 'product_category_name',
    'price': 'total_sales'
], inplace=True)

category_sales = category_sales.sort_values(
    by=['total_sales', 'product_category_name'],
    ascending=[False, True]
)
top9 = category_sales.head(9)
top9 = top9.reset_index(drop=True)
top9
```

product_category_name	total_sales
HEALTH BEAUTY	1258681.34
Watches present	1205005.68
bed table bath	1036988.68
sport leisure	988048.97
computer accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635290.85
housewares	632248.66
automotive	592720.11

Sql

```
-- Q3: Find the total sales per category
SELECT
    p.product_category,
    ROUND(SUM(oi.price), 2) AS total_sales
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category
ORDER BY total_sales DESC;
```

product category	total sales
HEALTH BEAUTY	1258681.34
watches present	1209005.68
bed table bath	1036988.68
sport leisure	388048.97
computer accessories	911954.32
Furniture Decoration	729762.49
Cool Stuff	635298.85
housewares	632258.66
automotive	592728.11
Garden tools	485256.46
toys	483986.60
babies	411764.89
perfumery	399124.87
telephony	323667.53
Furniture office	273960.70
stationary store	238943.23
PCS	222963.13
pet Shop	214325.42
musical instruments	193498.88
electrostyle	190048.58
hsk	179535.28
electronics	168286.74
games consoles	157465.22
Fashion Bags and Accessories	152823.54
Construction Tools	146672.59
Bags Accessories	140429.58
ELECTRICES 2	113337.74
Casa Construciao	83088.12
home appliances	80171.53
Agro Industria e Comercio	72530.47
Room Furniture	68916.56
fixed telephony	59583.80
skate confort	58572.84

% OF ORDERS PLACED IN INSTALLMENTS

python

```
installment_orders = payments[payments['payment_installments'] > 1]['order_id'].nunique()

total_orders = payments['order_id'].nunique()

percentage = (installment_orders / total_orders) * 100

percentage
```

51.46

Sql

```
-- Q4: Calculate the percentage of orders paid in installments
SELECT
    ROUND(
        100.0 * SUM(CASE WHEN payment_installments > 1 THEN 1 ELSE 0 END) / COUNT(*),
        2) AS installment_percentage
FROM payments;
```

```
=====
Q4:Percentage of orders paid in installments
=====
```

```
installment_percentage
| 51.46
```

NO. OF CUSTOMERS FROM EACH STATE

python

```
customers_state = customers['customer_state'].value_counts()  
  
customers_state
```

customer_state	
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380

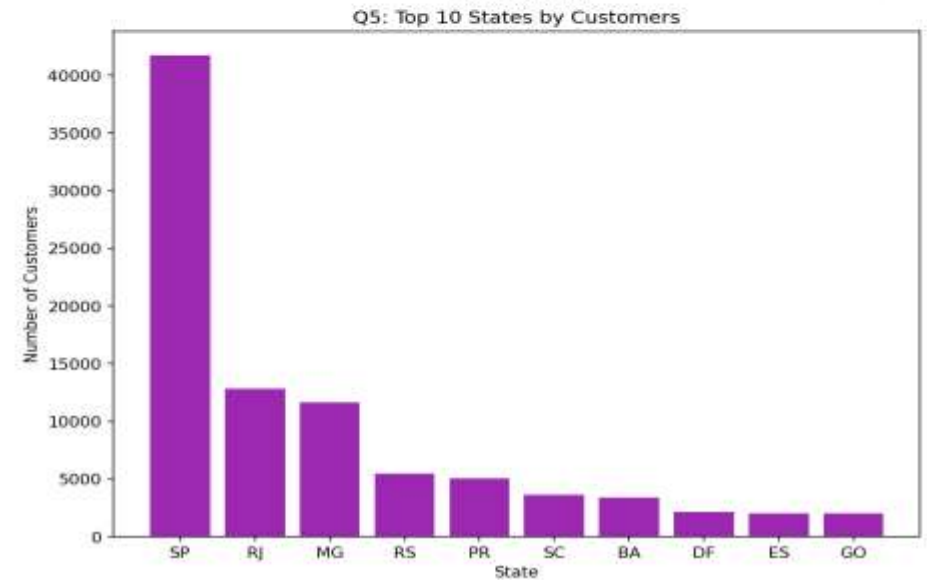
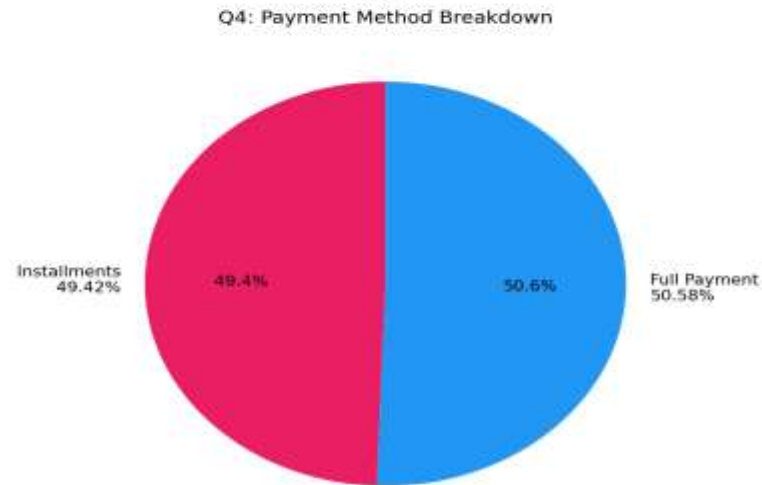
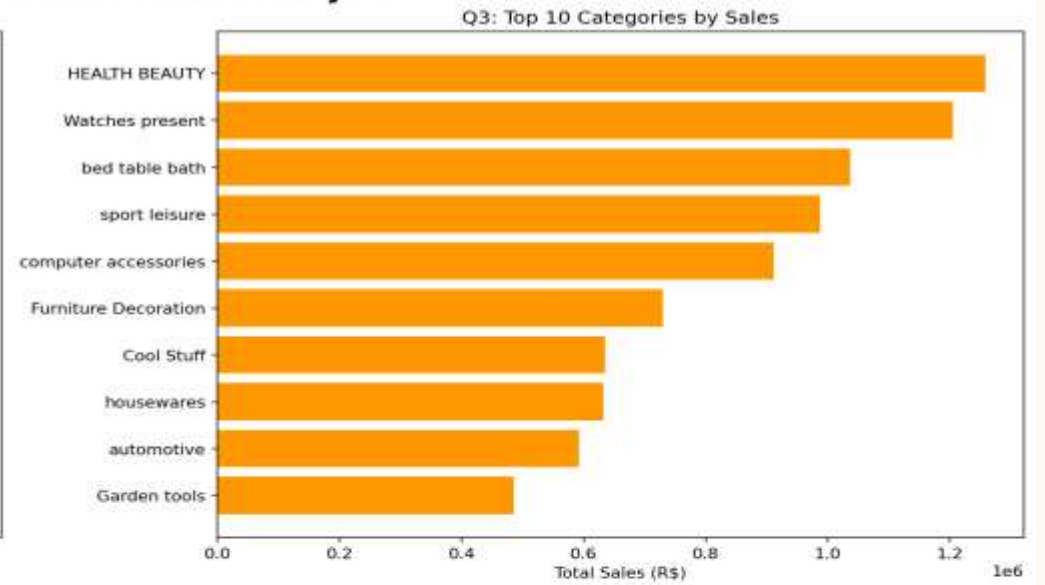
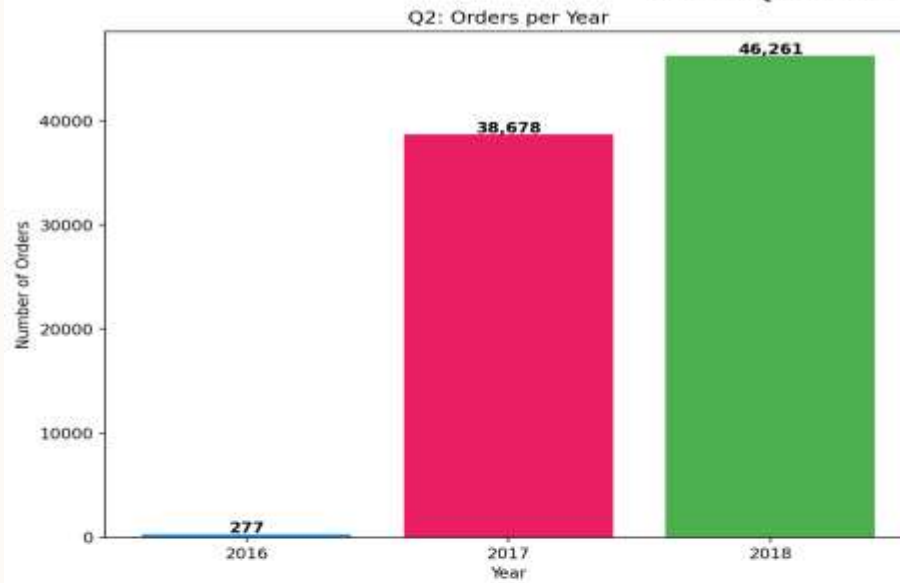
Sql

```
-- Q5: Count the number of customers from each state  
SELECT  
    customer_state,  
    COUNT(*) AS customer_count  
FROM customers  
GROUP BY customer_state  
ORDER BY customer_count DESC;
```

Q5: Number of customers from each state

customer_state	customer_count
SP	41746
RJ	12852
MG	11635
RS	5466
PR	5045
SC	3637
BA	3380
DF	2140
ES	2033
GO	2020
PE	1652
CE	1336
PA	975
MT	907
MA	747
MS	715
PB	536
PI	495
RN	485
AL	413
SE	350
TO	280
RO	253
AM	148
AC	81
AP	68
RR	46

Basic Queries — E-Commerce Analysis



Insights - Intermediate

Objective: Dive deeper into sales and order trends.

- Orders per month in 2018 – January peak 6,288 orders
- Avg products per order by city – Padre Carvalho, 7 products/orders
- Revenue % by product category – Health & Beauty 9.26%
- Price vs purchase frequency – Most purchased products, 527 times
- Total revenue by seller ranked – Top seller, Rs 2,29,472

NO. OF ORDERS PLACED IN 2018

python

```
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])
orders['month'] = orders['order_purchase_timestamp'].dt.to_period('M').astype(str)
orders_2018 = orders[orders['order_purchase_timestamp'].dt.year == 2018]
orders_count = orders_2018.groupby('month')['order_id'].count().reset_index()
orders_count.rename(columns={'order_id': 'total_orders'}, inplace=True)
orders_count = orders_count.sort_values('month')
orders_count = orders_count.reset_index(drop=True)
orders_count
```

month	total_orders
01	6288
02	5771
03	6164
04	5908
05	5883
06	5262
07	5408
08	5561
09	12
10	4

Sql

```
-- Q1: Calculate the number of orders per month in 2018
SELECT
    strftime('%m', order_purchase_timestamp) AS month,
    COUNT(*) AS total_orders
FROM orders
WHERE strftime('%Y', order_purchase_timestamp) = '2018'
GROUP BY month
ORDER BY month;
```

Q1: Number of orders per month in 2018

month	total_orders
01	6288
02	5771
03	6164
04	5908
05	5883
06	5262
07	5408
08	5561
09	12
10	4

AVG NO. OF PRODUCTS PER ORDER BY CUSTOMER CITY

python

```
products_per_order = order_items.groupby('order_id').size().reset_index(name='product_count')
merged = products_per_order.merge(
    orders[['order_id', 'customer_id']],
    on='order_id'
).merge(
    customers[['customer_id', 'customer_city']],
    on='customer_id'
)

result = merged.groupby('customer_city')['product_count'].mean().reset_index()

result['avg_products_per_order'] = result['product_count'].round(2)

result = result[['customer_city', 'avg_products_per_order']]
result = result.sort_values(
    by=['avg_products_per_order', 'customer_city'],
    ascending=[False, True]
)
result.head(10)
```

customer_city	avg_products_per_order
padre carvalho	7.0
celso ramos	6.5
candido godoi	6.0
datas	6.0
matias olimpio	5.0
cidelandia	4.0

Sql

```
-- Q2: Find the average number of products per order
-- grouped by customer city
SELECT
    c.customer_city,
    ROUND(AVG(oi.item_count), 2) AS avg_products_per_order
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN (
    SELECT order_id, COUNT(*) AS item_count
    FROM order_items
    GROUP BY order_id
) oi ON o.order_id = oi.order_id
GROUP BY c.customer_city
HAVING COUNT(*) > 10
ORDER BY avg_products_per_order DESC
LIMIT 20;
```

```
=====
Q2: Avg products per order by customer city (Top 20)
=====
customer_city  avg_products_per_order
padre carvalho      7.00
celso ramos         6.50
datas              6.00
candido godoi       6.00
matias olimpio      5.00
teixeira soares     4.00
picarra            4.00
morro de sao paulo  4.00
curralinho          4.00
cidelandia          4.00
inconfidentes       3.50
ipua               3.25
ubata              3.00
pedregulho         3.00
pacuja             3.00
ouvidor            3.00
nova esperanca do sul 3.00
jose boiteux        3.00
chapadao do lageado 3.00
capela             3.00
=====
```

REVENUE CONTRIBUTION OF EACH CATEGORY

python

```
temp = order_items.merge(
    products[['product_id', 'product_category']],
    on='product_id',
    how='inner'
)
category_revenue = temp.groupby('product_category')['price'].sum().reset_index()
category_revenue.rename(columns={
    'product_category': 'product_category_name',
    'price': 'revenue_category'
}, inplace=True)
total_revenue = category_revenue['revenue_category'].sum()

category_revenue['revenue_percentage'] = (
    category_revenue['revenue_category'] / total_revenue
) * 100

category_revenue['revenue_category'] = category_revenue['revenue_category'].round(2)
category_revenue['revenue_percentage'] = category_revenue['revenue_percentage'].round(2)

category_revenue = category_revenue.sort_values(
    by=['revenue_category', 'product_category_name'],
    ascending=[False, True]
)
category_revenue = category_revenue.reset_index(drop=True)
category_revenue
```

product_category_name	revenue_category	revenue_percentage
HEALTH BEAUTY	1258681.34	9.26
Watches present	1205005.68	8.87
bed table bath	1036988.68	7.63
sport leisure	988048.97	7.27
computer accessories	911954.32	6.71

Sql

```
-- Q3: Percentage of total revenue by each product category
SELECT
    p.product_category,
    ROUND(SUM(oi.price), 2) AS category_revenue,
    ROUND(100.0 * SUM(oi.price) / (SELECT SUM(price) FROM order_items), 2) AS revenue_percentage
FROM order_items oi
JOIN products p ON oi.product_id = p.product_id
GROUP BY p.product_category
ORDER BY revenue_percentage DESC;
```

Q3: Revenue percentage by product category		
product_category	category_revenue	revenue_percentage
HEALTH BEAUTY	1258681.34	9.26
Watches present	1205005.68	8.87
bed table bath	1036988.68	7.63
sport leisure	988048.97	7.27
computer accessories	911954.32	6.71
Furniture Decoration	729762.48	5.37
Good stuff	639290.85	4.87
housewares	622248.66	4.65
automotive	592720.11	4.36
Garden tools	485256.46	3.57
Toys	483946.60	3.56
hobbies	433764.89	3.23
perfumery	399174.82	2.98
telephony	328667.53	2.48
Furniture office	273960.20	2.02
stationary store	236943.23	1.78
PCS	222963.13	1.64
pet shop	214335.41	1.58
musical instruments	193498.88	1.41
electrostatic	198546.58	1.46
ham	179235.28	1.32
electronics	160246.74	1.18
Games consoles	157405.22	1.16
fashion bags and accessories	152823.54	1.12
Construction Tools construction	144672.59	1.06
bags Accessories	140429.88	1.02
ELECTRICALS 2	113317.70	0.83
Cable Construction	81088.12	0.61
home appliances	80171.53	0.59
Agro industria e Comercio	72530.47	0.53
modern furniture	68936.56	0.51
Fixed telephony	59563.00	0.44

CORRELATION : PRODUCT PRICE VS PURCHASE FREQUENCY

python

```
# Frequency
product_freq = order_items.groupby('product_id').size().reset_index(name='purchase_count')

# Avg price
product_price = order_items.groupby('product_id')['price'].mean().reset_index()

# Merge
merged = product_freq.merge(product_price, on='product_id')

# Correlation
correlation = merged['price'].corr(merged['purchase_count'])

correlation
```

```
np.float64(-0.032139862680945167)
```

Sql

```
--Q4: correlation: product Price vs Purchase Frequency
Select
(
    count(*) * sum(avg_price * purchase_count) /
    sum(avg_price) * sum(purchase_count)
) /
SQRT(
    (count(*) * sum(avg_price * avg_price) - POWER(sum(avg_price), 2)) *
    (count(*) * sum(purchase_count * purchase_count) - POWER(sum(purchase_count), 2))
) as correlation
FROM (
    select
    product_id,
    count(*) as purchase_count,
    avg(price) as avg_price
    from order_items
    group by product_id
) t;
```

```
-0.032139862680945167
```


REVENUE BY SELLER AND RANKING

python

```
seller_revenue = order_items.groupby('seller_id')['price'].sum().reset_index()
seller_revenue.rename(columns={'price': 'revenue'}, inplace=True)

seller_revenue['revenue'] = seller_revenue['revenue'].round(6)

seller_revenue = seller_revenue.sort_values(
    by=['revenue', 'seller_id'],
    ascending=[False, True]
)

seller_revenue['rank'] = seller_revenue['revenue'].rank(
    method='dense',
    ascending=False
).astype(int)

seller_revenue = seller_revenue.reset_index(drop=True)
seller_revenue.head(10)
```

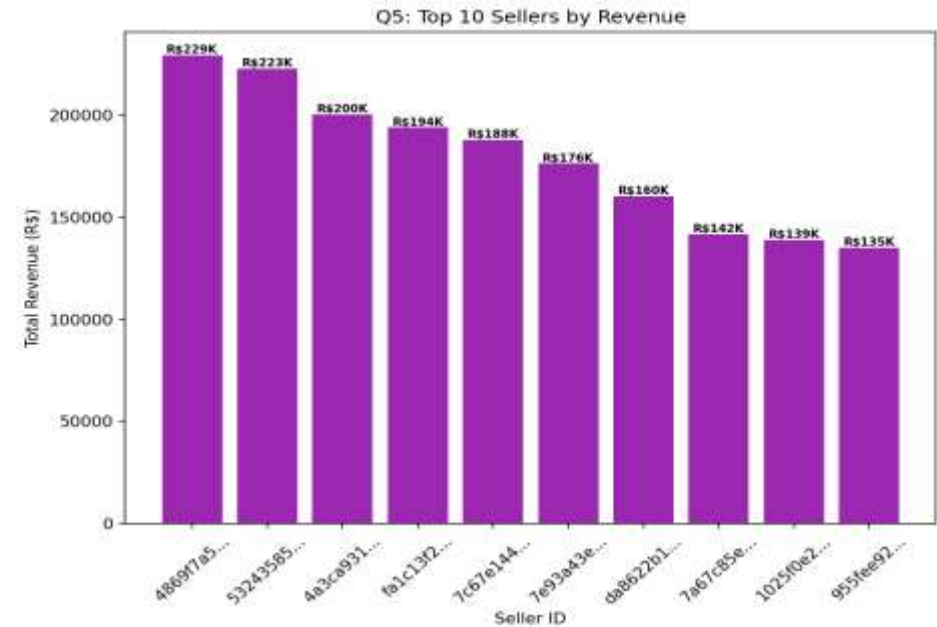
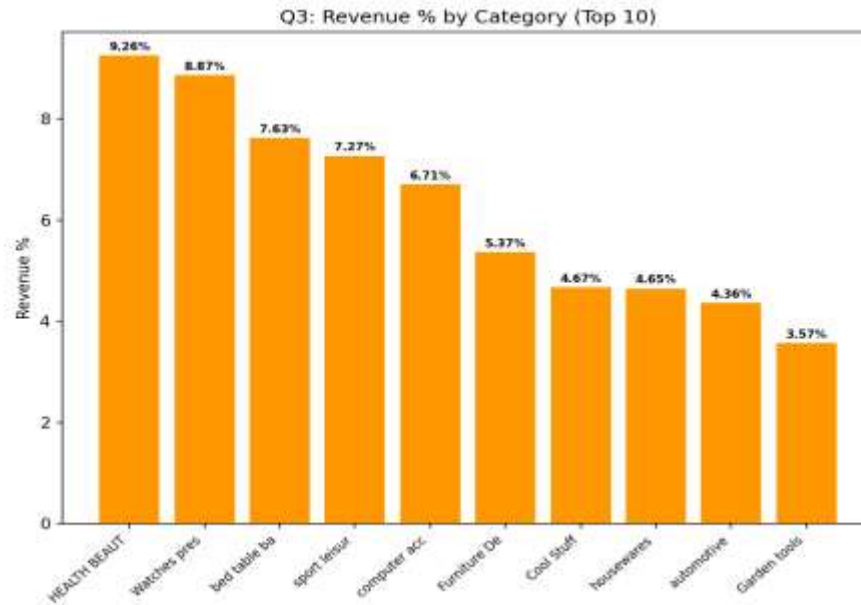
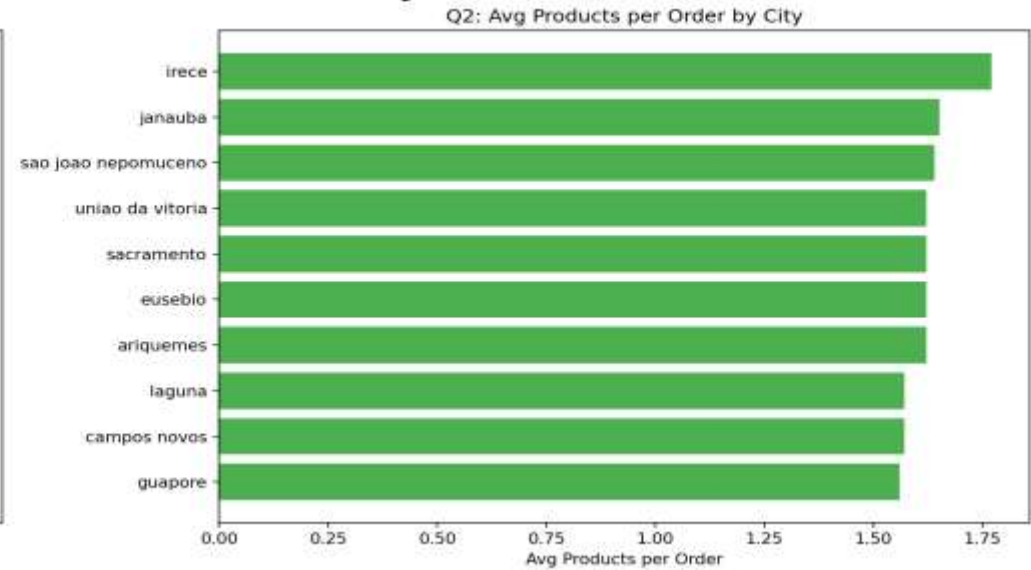
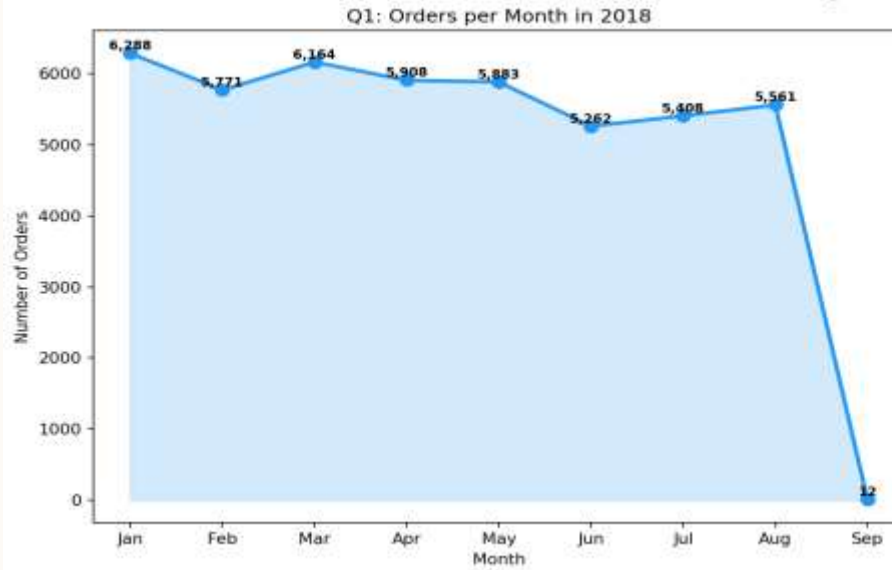
4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1
53243585a1d6dc2643021fd1853d8905	222776.05	2
4a3ca9315b744ce9f8e9374361493884	200472.92	3
fa1c13f2614d7b5c4749cbc52fecda94	194042.03	4
7c67e1448b00f6e969d365cea6b010ab	187923.89	5
7e93a43ef30c4f03f38b393420bc753a	176431.87	6
da8622b14eb17ae2831f4ac5b9dab84a	160236.57	7
7a67c85e85bb2ce8582c35f2203ad736	141745.53	8
1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	9
955fee9216a65b617aa5c0531780ce60	135171.70	10

Sql

```
-- Q5: Total revenue by each seller ranked by revenue
SELECT
    oi.seller_id,
    ROUND(SUM(oi.price), 2) AS total_revenue,
    RANK() OVER (ORDER BY SUM(oi.price) DESC) AS revenue_rank
FROM order_items oi
GROUP BY oi.seller_id
ORDER BY revenue_rank
LIMIT 20;
```

```
=====
Q5: Total revenue by seller - Top 20
=====
seller_id  total_revenue  revenue_rank
4869f7a5dfa277a7dca6462dcf3b52b2      229472.63      1
53243585a1d6dc2643021fd1853d8905      222776.05      2
4a3ca9315b744ce9f8e9374361493884      200472.92      3
fa1c13f2614d7b5c4749cbc52fecda94      194042.03      4
7c67e1448b00f6e969d365cea6b010ab      187923.89      5
7e93a43ef30c4f03f38b393420bc753a      176431.87      6
da8622b14eb17ae2831f4ac5b9dab84a      160236.57      7
7a67c85e85bb2ce8582c35f2203ad736      141745.53      8
1025f0e2d44d7041d6cf58b6550e0bfa      138968.55      9
955fee9216a65b617aa5c0531780ce60      135171.70     10
46dc3b2cc0980fb8ec44634e21d2718e      128111.19     11
6560211a19b47992c3666cc44a7e94ce      123304.83     12
620c87c171fb2a6dd6e8bb4dec959fc6      114774.50     13
7d13fca15225358621be4086e1eb0964      113628.97     14
5dceca129747e92ff8ef7a997dc4f8ca      112155.53     15
1f50f920176fa81dab994f9023523100      106939.21     16
cc419e0650a3c5ba77189a1882b7556a      104288.42     17
a1043baf4d71dff536d0c462352beb48      101901.16     18
3d871de0142ce09b7081e2b9d1733cb1      94914.20      19
edb1ef5e36e0c8cd84eb3c9b003e486d       79284.55      20
=====
```

Intermediate Queries — E-Commerce Analysis



Insights - Advanced

Objective: Generate strategic and customer-centric insights.

- Moving average of order values – Tracks customer spending trends
- Cumulative sales per month – 2018 hit, Rs 6.35 by August
- Year-over-year growth rate – 2017 grew 12,745% vs 2016
- Customer retention rate – 0% most buy only once
- Top 3 customers per year – 2017 top spender, Rs 13,440

MOVING AVERAGE OF ORDER VALUES FOR EACH CUSTOMER OVER THEIR ORDER HISTORY

python

```
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

order_value = order_items.groupby('order_id')['price'].sum().reset_index()
order_value.rename(columns={'price': 'order_value'}, inplace=True)

order_value = order_value.merge(
    orders[['order_id', 'customer_id', 'order_purchase_timestamp']],
    on='order_id',
    how='inner'
)

order_value = order_value.sort_values(['customer_id', 'order_purchase_timestamp'])

order_value['moving_avg'] = order_value.groupby('customer_id')['order_value'] \
    .transform(lambda x: x.rolling(window=3, min_periods=1).mean())

order_value = order_value[['customer_id', 'order_id', 'order_purchase_timestamp', 'order_value', 'moving_avg']]

order_value = order_value.reset_index(drop=True)
order_value.head(10)
```

customer_id	order_id	order_purchase_timestamp	order_value	moving_avg
00012a2ce0f8dcd20d59ce98491703	5793e5c0931d531fa42589eb83b9dafe	2017-11-14 16:08:26	89.80	89.80
000161a05a660d5901f007fab4c27140	a44895d395d7e0702b6a162fa2dbeced	2017-07-16 09:40:32	54.90	54.90
0001d6190eda81894bca3d49ed079	316a104623542e4d75189bc372bc5f8d	2017-02-28 11:06:43	179.99	179.99
000241489344307404f0ace7a26f1d5	5825ce2e88d5346438686c0ba99e5ee	2017-08-16 13:09:20	149.90	149.90
000379cdec625522490c315e70c7a9fb	0ab7fb080864af9141453c91878ed7a	2018-04-02 13:42:17	93.00	93.00

Sql

```
-- Q1: Moving average of order values for each customer
SELECT
    o.customer_id,
    o.order_id,
    o.order_purchase_timestamp,
    ROUND(SUM(oi.price), 2) AS order_value,
    ROUND(AVG(SUM(oi.price)) OVER (
        PARTITION BY o.customer_id
        ORDER BY o.order_purchase_timestamp
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
    ), 2) AS moving_avg
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
GROUP BY o.order_id, o.customer_id, o.order_purchase_timestamp
ORDER BY o.customer_id, o.order_purchase_timestamp
LIMIT 20;
```

Q1: Moving Average of Order Values per Customer				
mp	order_value	moving_avg	customer_id	order_id order_purchase_timestamp
00012a2ce0f8dcd20d59ce98491703	89.80	89.80	5793e5c0931d531fa42589eb83b9dafe	2017-11-14 16:08:26
000161a05a660d5901f007fab4c27140	54.90	54.90	a44895d395d7e0702b6a162fa2dbeced	2017-07-16 09:40:32
0001d6190eda81894bca3d49ed079	179.99	179.99	316a104623542e4d75189bc372bc5f8d	2017-02-28 11:06:43
000241489344307404f0ace7a26f1d5	149.90	149.90	5825ce2e88d5346438686c0ba99e5ee	2017-08-16 13:09:20
000379cdec625522490c315e70c7a9fb	93.00	93.00	0ab7fb080864af9141453c91878ed7a	2018-04-02 13:42:17
000419c5494106c386a07b5635748086	34.30	34.30	07fec3baf9ac80865b60f640c4f923c6	2018-03-02 17:47:40
00046a566d407e99b969756e0b18f282	120.90	120.90	8c3d752c5c92227878fae49aeaddbfdf	2017-12-18 11:08:30
00050bf6e01e69d5c0fd612f1bcfb09c	69.99	69.99	fa08ef338ce030a084d0945b3832e431	2017-09-17 16:04:44
000598caf2ef4117407665ac31275130	1107.00	1107.00	9b961b804e797f63622137ff79b1c1af	2018-08-11 12:14:35
0005ae7fb0696d34b3424dc0da0e9fd0	134.90	134.90	263ba12390d0fbce329dd16da8cd70f8	2018-06-29 9:46:53
00062b33cb9f6fe976afdcff967ea74d	47.99	47.99	c208db5638f7f1cd0d4d185856852f864	2017-03-15 23:44:00
00066ccbe787a588c52bd5ffa04500e3	189.60	189.60	728416bedb65935dbf78a0cc0308d6f8	2018-02-06 16:10:00

COMULATIVE SALES PER MONTH OF EACH YEAR

python

```
temp = orders.merge(order_items, on='order_id', how='inner')
temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])
temp['year'] = temp['order_purchase_timestamp'].dt.year
temp['month'] = temp['order_purchase_timestamp'].dt.month
temp['month_name'] = temp['order_purchase_timestamp'].dt.month_name
monthly_sales = temp.groupby(['year', 'month', 'month_name'])['price'].sum().reset_index()
monthly_sales = monthly_sales.sort_values(['year', 'month'])

monthly_sales['cumulative_sales'] = monthly_sales.groupby('year')['price'].cumsum()
monthly_sales.rename(columns={'price': 'monthly_sales'}, inplace=True)
monthly_sales = monthly_sales[
    ['year', 'month', 'month_name', 'monthly_sales', 'cumulative_sales']]
monthly_sales = monthly_sales.reset_index(drop=True)

monthly_sales['monthly_sales'] = monthly_sales['monthly_sales'].round(6)
monthly_sales['cumulative_sales'] = monthly_sales['cumulative_sales'].round(6)
monthly_sales.head(10)
```

year	month	month_name	monthly_sales	cumulative_sales
2016	9	September	267.36	267.36
2016	10	October	49507.66	49775.02
2016	12	December	10.90	49785.92
2017	1	January	120312.87	120312.87
2017	2	February	247303.02	367615.89

Sql

```
-- Q2: Cumulative sales per month for each year
SELECT
    strftime('%Y', o.order_purchase_timestamp) AS year,
    strftime('%m', o.order_purchase_timestamp) AS month,
    ROUND(SUM(oi.price), 2) AS monthly_sales,
    ROUND(SUM(SUM(oi.price)) OVER (
        PARTITION BY strftime('%Y', o.order_purchase_timestamp)
        ORDER BY strftime('%m', o.order_purchase_timestamp)
    ), 2) AS cumulative_sales
FROM orders o
JOIN order_items oi ON o.order_id = oi.order_id
WHERE o.order_purchase_timestamp IS NOT NULL
GROUP BY year, month
ORDER BY year, month;
```

Q2: Cumulative Sales per Month for Each Year			
year	month	monthly_sales	cumulative_sales
NaN	NaN	1885407.74	1885407.74
2016	09	207.86	207.86
2016	10	41119.73	41327.59
2016	12	10.90	41338.49
2017	01	105755.33	105755.33
2017	02	217569.07	323324.40
2017	03	321677.99	645002.39
2017	04	313577.07	958579.46
2017	05	435006.74	1393586.20
2017	06	379064.04	1772650.24
2017	07	433855.96	2206506.20
2017	08	498744.61	2705250.81
2017	09	529777.98	3235028.79
2017	10	570711.41	3805740.20
2017	11	863832.85	4669573.05
2017	12	640513.15	5310086.20
2018	01	831217.39	831217.39
2018	02	724462.30	1555679.69
2018	03	845601.89	2401281.58
2018	04	842773.14	3244054.72
2018	05	860456.40	4104511.12
2018	06	745952.12	4850463.24
2018	07	769345.97	5619811.21
2018	08	735000.06	6354811.27

YEAR ON YEAR GROWTH

python

```
temp = orders.merge(order_items, on='order_id', how='inner')
temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])
temp['year'] = temp['order_purchase_timestamp'].dt.year
yearly_sales = temp.groupby('year')['price'].sum().reset_index()
yearly_sales.rename(columns={'price': 'total_sales'}, inplace=True)

yearly_sales = yearly_sales.sort_values('year')

yearly_sales['previous_year_sales'] = yearly_sales['total_sales'].shift(1)

yearly_sales['yoy_growth_percentage'] = (
    (yearly_sales['total_sales'] - yearly_sales['previous_year_sales']) /
    yearly_sales['previous_year_sales']
) * 100

yearly_sales['total_sales'] = yearly_sales['total_sales'].round(6)
yearly_sales['previous_year_sales'] = yearly_sales['previous_year_sales'].round(6)
yearly_sales['yoy_growth_percentage'] = yearly_sales['yoy_growth_percentage'].round(2)

yearly_sales = yearly_sales.reset_index(drop=True)
yearly_sales
```

year	total_sales	previous_year_sales	yoy_growth_percentage
2016	49785.92	NaN	NaN
2017	6155806.98	49785.92	12264.55
2018	7386050.80	6155806.98	19.99

Sql

```
-- Q3: Year-over-year growth rate of total sales
SELECT
    year,
    total_sales,
    LAG(total_sales) OVER (ORDER BY year) AS prev_year_sales,
    ROUND(100.0 * (total_sales - LAG(total_sales) OVER (ORDER BY year))
        / LAG(total_sales) OVER (ORDER BY year), 2) AS yoy_growth_rate
FROM (
    SELECT
        strftime('%Y', o.order_purchase_timestamp) AS year,
        ROUND(SUM(oi.price), 2) AS total_sales
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_purchase_timestamp IS NOT NULL
    GROUP BY year
) yearly_sales
ORDER BY year;
```

Q3: Year-over-Year Growth Rate of Total Sales

year	total_sales	prev_year_sales	yoy_growth_rate
NaN	1885407.74	NaN	NaN
2016	41338.49	1885407.74	-97.81
2017	5310086.20	41338.49	12745.38
2018	6354811.27	5310086.20	19.67

CUSTOMER RETENTION RATE

python

```
# Ensure datetime
orders['order_purchase_timestamp'] = pd.to_datetime(orders['order_purchase_timestamp'])

# Sort orders
orders_sorted = orders.sort_values(['customer_id', 'order_purchase_timestamp'])

# First purchase
first_order = orders_sorted.groupby('customer_id')['order_purchase_timestamp'].first().reset_index()

# Merge
orders_merged = orders_sorted.merge(first_order, on='customer_id', suffixes=('', '_first'))

# Days difference
orders_merged['diff_days'] = (orders_merged['order_purchase_timestamp'] -
                             orders_merged['order_purchase_timestamp_first']).dt.days

# Retained customers (within 180 days)
retained_customers = orders_merged[
    (orders_merged['diff_days'] > 0) & (orders_merged['diff_days'] <= 180)
]['customer_id'].nunique()

total_customers = orders['customer_id'].nunique()

retention_rate = (retained_customers / total_customers) * 100

retention_rate
```

0.0

Sql

```
-- Q4: Retention rate - customers who purchase again within 6 months
SELECT
    ROUND(100.0 * COUNT(DISTINCT returning.customer_id) /
          COUNT(DISTINCT first_purchase.customer_id), 2) AS retention_rate
FROM (
    SELECT customer_id, MIN(order_purchase_timestamp) AS first_date
    FROM orders
    WHERE order_purchase_timestamp IS NOT NULL
    GROUP BY customer_id
) first_purchase
LEFT JOIN (
    SELECT DISTINCT o1.customer_id
    FROM orders o1
    JOIN orders o2 ON o1.customer_id = o2.customer_id
    WHERE o1.order_purchase_timestamp > o2.order_purchase_timestamp
    AND julianday(o1.order_purchase_timestamp) -
        julianday(o2.order_purchase_timestamp) <= 180
) returning ON first_purchase.customer_id = returning.customer_id;
```

Q4: Customer Retention Rate (within 6 months)

retention_rate
0.0

TOP 3 CUSTOMERS WHO SPENT THE MOST MONEY IN EACH YEAR.

python

```
temp = orders.merge(order_items, on='order_id', how='inner')
temp['order_purchase_timestamp'] = pd.to_datetime(temp['order_purchase_timestamp'])

temp['year'] = temp['order_purchase_timestamp'].dt.year
result = temp.groupby(['year', 'customer_id'])['price'].sum().reset_index()
result.rename(columns={'price': 'total_spent'}, inplace=True)
result['rnk'] = result.groupby('year')['total_spent'] \
    .rank(method='min', ascending=False).astype(int)

result = result[result['rnk'] <= 3]
result = result[['year', 'customer_id', 'total_spent', 'rnk']]
result = result.sort_values(['year', 'rnk', 'customer_id'])
result = result.reset_index(drop=True)
result
```

year	customer_id	total_spent	rnk
2016	a9dc96b027d1252bbac0a9b72d837fc6	1399.00	1
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	2
2016	4a06381959b6670756de02e07b83815f	1199.00	3
2017	1617b1357756262bfa56ab541c47bc16	13440.00	1
2017	c6e2731c5b391845f6800c97401a43a9	6735.00	2
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499.00	3
2018	ec5b2ba62e574342386871631fafd3fc	7160.00	1
2018	f48d464a0baaea338cb25f816991ab1f	6729.00	2
2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.90	3

Sql

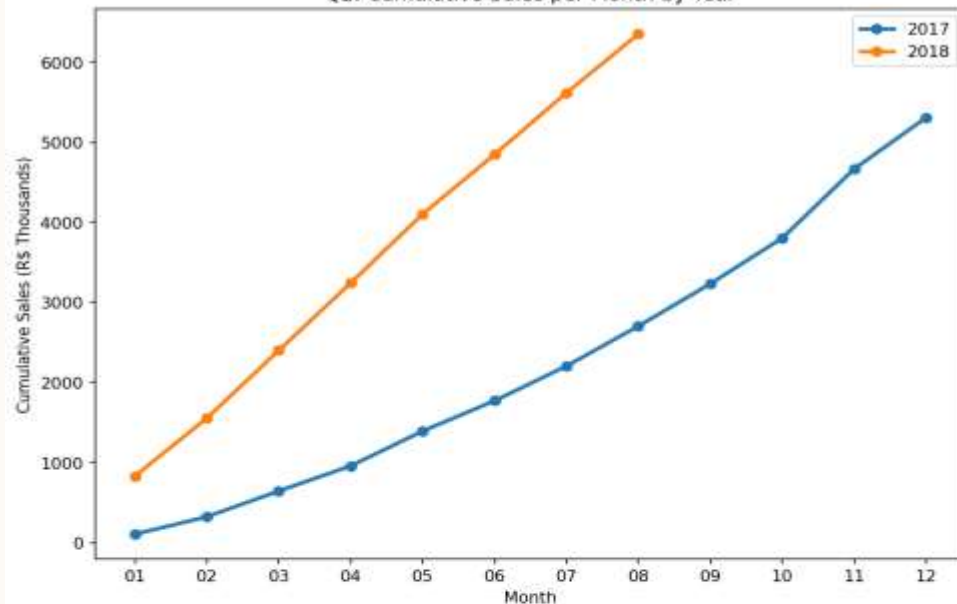
```
-- Q5: Top 3 customers who spent most money each year
SELECT year, customer_id, total_spent, year_rank
FROM (
    SELECT
        strftime('%Y', o.order_purchase_timestamp) AS year,
        o.customer_id,
        ROUND(SUM(oi.price), 2) AS total_spent,
        RANK() OVER (
            PARTITION BY strftime('%Y', o.order_purchase_timestamp)
            ORDER BY SUM(oi.price) DESC
        ) AS year_rank
    FROM orders o
    JOIN order_items oi ON o.order_id = oi.order_id
    WHERE o.order_purchase_timestamp IS NOT NULL
    GROUP BY year, o.customer_id
)
WHERE year_rank <= 3
ORDER BY year, year_rank;
```

Q5: Top 3 Customers by Spending per Year

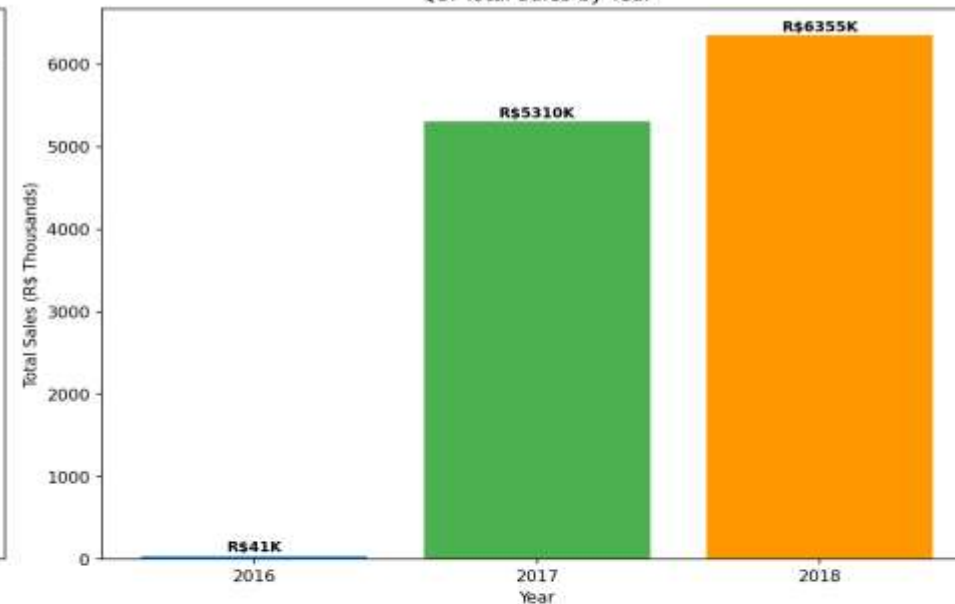
year	customer_id	total_spent	year_rank
NaN	e9b0d0eb3015ef1c9ce6cf5b9dcbee9f	4059.00	1
NaN	46bb3c0b1a65c8399d0363cefbcc4f37	3124.00	2
NaN	7e7d2271d6f55b03b0bd4615c5f2de6e	3105.00	3
2016	1d34ed25963d5aae4cf3d7f3a4cda173	1299.99	1
2016	4a06381959b6670756de02e07b83815f	1199.00	2
2016	85f0e92957e9fb9c5f72ba5378f492a0	949.99	3
2016	1fc56719b52f82c03caddc5faf531fbb	949.99	3
2017	1617b1357756262bfa56ab541c47bc16	13440.00	1
2017	c6e2731c5b391845f6800c97401a43a9	6735.00	2
2017	3fd6777bbce08a352fddd04e4a7cc8f6	6499.00	3
2018	ec5b2ba62e574342386871631fafd3fc	7160.00	1
2018	f48d464a0baaea338cb25f816991ab1f	6729.00	2
2018	e0a2412720e9ea4f26c1ac985f6a7358	4599.90	3

Advanced Queries — E-Commerce Analysis

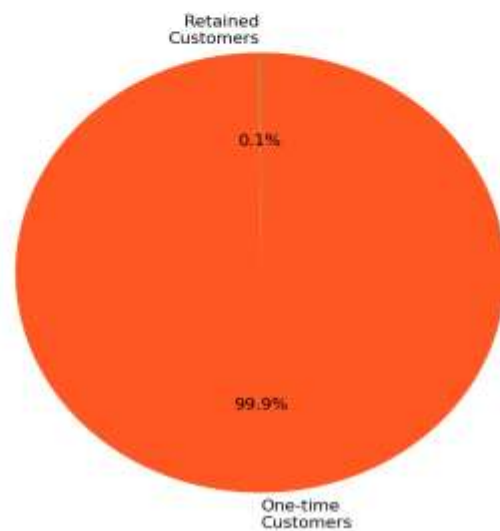
Q2: Cumulative Sales per Month by Year



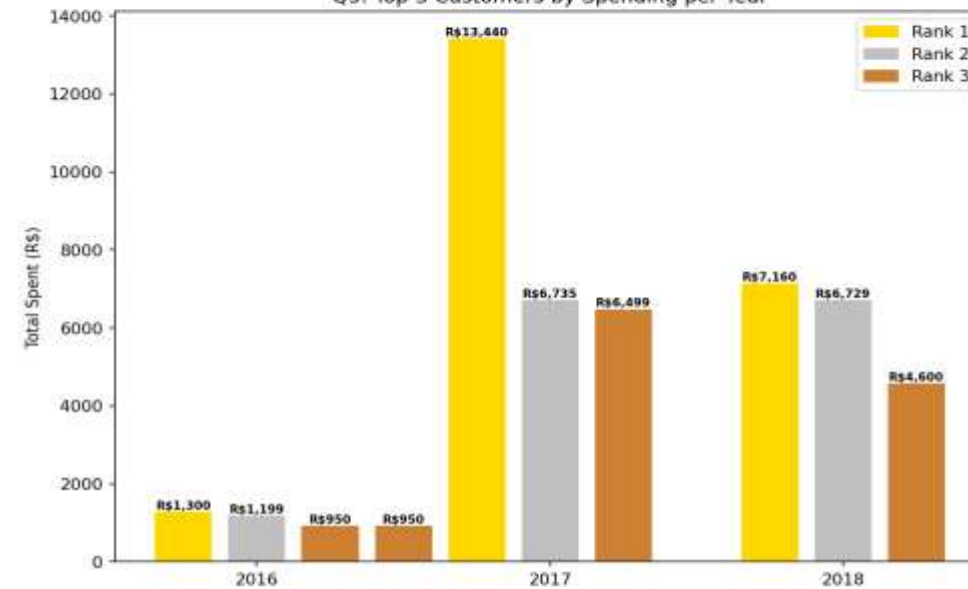
Q3: Total Sales by Year



Q4: Customer Retention Rate (6 months)



Q5: Top 3 Customers by Spending per Year



Key Business Insight

- 1. São Paulo dominates — SP state has 42% of all customers
- 2. Health & Beauty is top category — Rs1.25M in total sales
- 3. Half of customers pay in installments — 49.42% use installment plans
- 4. Massive growth in 2017 — Business grew 12,745% compared to 2016
- 5. Low retention rate — Most customers purchase only once, suggesting need for loyalty programs
- 6. Seasonal peaks — November 2017 had highest monthly sales (Rs863K) likely due to Black Friday



Thank You